Aplicações Web e Desktop para Consulta de Dados de Servidores do Executivo Federal

Pedro Machado[1], **Marcos Freitas**[1], **Alexsander Santos**[1]

[1]Universidade Federal do Rio Grande (FURG)
Sistemas para Internet II – Rio Grande – RS – Brasil

pedro.machado@furg.br, mcopellofreitas@hotmail.com

alexsander.oml@gmail.com

Abstract. *This report presents web and desktop applications for querying Brazilian federal employee data. It describes the team's work, the two sprints, the implementation and the tests. PostgreSQL, Flask with Gunicorn and React run in separate containers. A Python/Tkinter desktop client consumes the same HTTP API, with separate Windows and Linux executables. The report also records the remaining limits of data import, interface testing and concurrent searches.*

Resumo. *Este relatório apresenta aplicações web e desktop para consulta de dados de servidores públicos federais. São descritos o trabalho da equipe, as duas sprints, a implementação e os testes. PostgreSQL, Flask com Gunicorn e React são executados em containers separados. O cliente Python/Tkinter usa a mesma API HTTP e possui executáveis para Windows e Linux. O texto também registra os limites de importação dos dados, de testes da interface e de buscas simultâneas.*

1. Objetivo e divisão do trabalho

A aplicação consulta informações dos conjuntos de Aposentados e Carreiras/Cargos do Portal Brasileiro de Dados Abertos [Portal Brasileiro de Dados Abertos 2026a, Portal Brasileiro de Dados Abertos 2026b]. O trabalho foi desenvolvido na disciplina Sistemas para Internet II, com GitHub, Docker, PostgreSQL, Python, Flask e Gunicorn, conforme o enunciado [Sistemas para Internet II 2026].

Pedro Machado ficou responsável pela infraestrutura, pelos testes e pelo relatório. Marcos Freitas trabalhou no banco de dados e na API. Alexsander Santos desenvolveu as interfaces web e desktop. As mudanças foram agrupadas por entregas: ajustes pequenos relacionados ficaram juntos, e etapas maiores foram separadas.

2. Sprints e entregas

A primeira sprint ocorreu de 12 a 26/09/2026. A segunda ocorreu de 27/09 a 20/10/2026, incluindo a conclusão das entregas e os testes integrados. As 70 tarefas foram mapeadas no Jira: 36 na primeira sprint e 34 na segunda, incluindo o cliente desktop (PROJ-71). As alterações usam Conventional Commits e branches por entrega, integradas por Pedro em `dev`. Os dias sem entregas aparecem como trechos horizontais no burndown. A distribuição das atividades e os commits de cada integrante estão na Tabela 1. A organização inicial das tarefas foi realizada em 13/09.

Tabela 1. Entregas da equipe por dia e integrante.

Dia	Integrante	Entrega	Commit
01/09	Pedro Machado	criação do repositório	`b891da4`
12/09	Pedro Machado	estrutura inicial e pastas da aplicação	`a52d14c`
13/09	Marcos Freitas	dependências, download e análise dos datasets	`5305ddd`
13/09	Alexsander Santos	primeira tela com React, Vite e Tailwind	`1c5609b`
14/09	Pedro Machado	compose e imagem do postgres	`f0c4b65`
15/09	Alexsander Santos	formulário de busca e layout da página	`57772a6`
16/09	Marcos Freitas	schema e mapeamento dos campos	`6b46d5a`
17/09	Pedro Machado	imagens do backend e do frontend	`3160cb7`
19/09	Marcos Freitas	tratamento dos arquivos e duplicados entre competências	`0087fec`
19/09	Alexsander Santos	tabela de resultados com dados de exemplo	`2e64211`
20/09	Marcos Freitas	importação, preparo inicial do banco e índices	`dd4c930`
20/09	Pedro Machado	variáveis de ambiente e validação do compose	`97e3425`
20/09	Pedro Machado	PostgreSQL com schema e índices na inicialização	`69f046f`
21/09	Alexsander Santos	paginação da lista e estados sem resultado	`ba10650`
22/09	Marcos Freitas	Flask, Gunicorn e conexão com PostgreSQL	`25edf71`
23/09	Pedro Machado	plano de testes e casos de SQL injection	`e85668e`
24/09	Marcos Freitas	consultas por nome, cargo, UF e órgão	`f72bbb0`
25/09	Alexsander Santos	busca e resultados no celular	`c8b2b99`
27/09	Marcos Freitas	combinação dos filtros na consulta	`90d307f`
27/09	Alexsander Santos	busca por nome ligada à API	`1f68851`
28/09	Pedro Machado	integração dos containers com a carga local	`2a6f83e`
29/09	Marcos Freitas	similaridade e paginação das consultas	`9e41227`
30/09	Marcos Freitas	validação dos parâmetros de busca	`9eff8be`
30/09	Alexsander Santos	opção de busca por nomes parecidos	`5f00285`
01/10	Pedro Machado	reinício dos containers, limite de buscas e timeout	`7311969`
02/10	Alexsander Santos	filtros combinados no formulário	`c5e027e`
03/10	Marcos Freitas	tratamento de erros e respostas JSON	`67bdb31`
03/10	Alexsander Santos	navegação entre páginas e resultado único	`d239861`
04/10	Marcos Freitas	endpoints e índice de similaridade	`0556db0`
04/10	Alexsander Santos	avisos da busca e layout em telas pequenas	`169dfb8`
04/10	Pedro Machado	carga e sobrecarga da api	`8b6c2db`
04/10	Pedro Machado	cenários de erro dos endpoints	`6334ce7`
05/10	Pedro Machado	sql injection e integridade do banco	`dab9c10`
06/10	Marcos Freitas	filtros inválidos e exemplos dos endpoints	`b25829d`
07/10	Alexsander Santos	busca vazia e respostas inesperadas	`8230b59`
09/10	Pedro Machado	inicialização local sem arquivo .env	`2988d06`
11/10	Alexsander Santos	cliente HTTP do desktop e testes das respostas	`4195461`
12/10	Alexsander Santos	janela Tkinter com filtros e paginação	`bf4adc0`
13/10	Pedro Machado	compilação do desktop para Windows e Linux	`6610646`
13/10	Pedro Machado	execução local e carga dos dados	`a22d123`
18/10	Marcos Freitas	correção dos padrões com curinga	`9be7ade`
18/10	Alexsander Santos	estados da busca, UF ausente e ícone	`8bb1669`
18/10	Alexsander Santos	atualização da dependência de build	`6e49b8b`
19/10	Marcos Freitas	importação com as variáveis do Compose	`8e968e1`
19/10	Pedro Machado	verificação do Tkinter antes de compilar	`ad62889`
20/10	Pedro Machado	persistência do banco ao recriar os containers	`bfbcd21`
20/10	Alexsander Santos	atualização de segurança do Vite	`32aa95e`
20/10	Pedro Machado	relatório final e instruções de execução	–

2.1. Burndown

Os gráficos usam os Story Points das tarefas mapeadas no Jira e as entregas do projeto. A Sprint 1 soma 56 pontos. A Sprint 2 começa com 59, incluindo os 2 pontos do desktop, e recebe mais 2 pontos para a finalização do relatório (PROJ-72), totalizando 61. Assim, o trabalho reúne 117 pontos. Cada tarefa é contada uma única vez; os ajustes de uma entrega não geram pontos adicionais. O saldo considera a conclusão dos filtros, do desktop e da documentação ao longo da segunda sprint. Os pontos são baixados na entrega de cada atividade, não na última revisão de seu conteúdo. A finalização do relatório permanece em 20/10; as revisões não contabilizam os mesmos pontos novamente. A linha tracejada é uma referência de redução uniforme dos pontos iniciais até o encerramento de cada sprint.

A divisão de tarefas e o gráfico foram reunidos em 08/10; a análise da equipe, em 09/10. O desktop foi entregue em 13/10. Essas atividades têm suas próprias baixas, separadas da finalização do PDF.

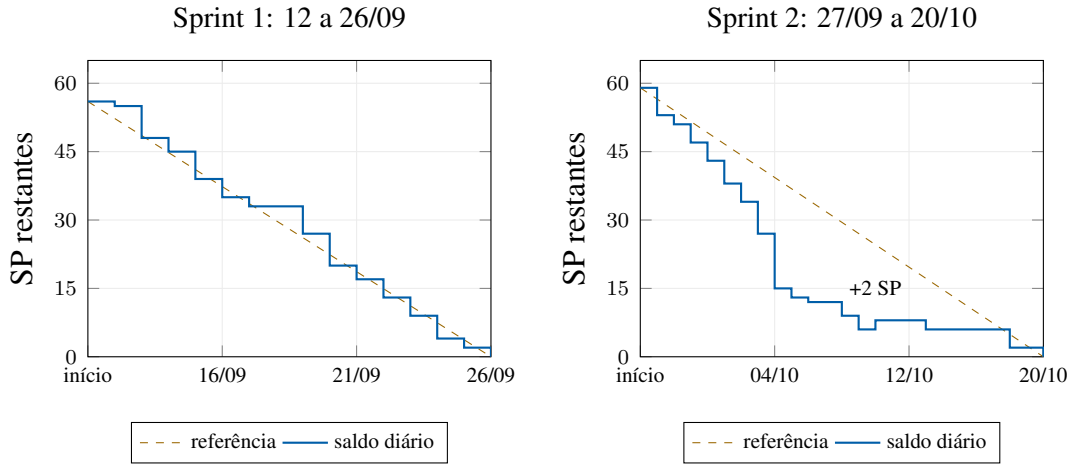


Figura 1. Burndown das duas sprints. O acréscimo de 2 SP em 10/10 corresponde à finalização do relatório (PROJ-72).

3. Implementação

O Compose inicia três serviços: PostgreSQL, backend Flask/Gunicorn e frontend React/Vite. A interface fica na porta 5173 e a API, na 8000. As requisições da tela passam pelo proxy do Vite até o backend. O comando de execução, na raiz, é `docker compose up --build` [Docker 2026].

3.1. Banco e preparação dos dados

Foram analisadas as competências de setembro de 2025 a agosto de 2026. O mapeamento seguiu os dicionários de Aposentados e de Carreiras/Cargos, publicados pelo Ministério da Gestão e da Inovação em Serviços Públicos [Ministério da Gestão e da Inovação em Serviços Públicos 2018, Ministério da Gestão e da Inovação em Serviços Públicos 2019]. Na inspeção inicial, os arquivos de agosto estavam indisponíveis, e o arquivo de Aposentados de dezembro de 2025 estava vazio.

Os scripts leem os dados em Latin-1, removem espaços e bytes NUL, verificam colunas e campos obrigatórios e descartam duplicados entre os arquivos do período. CPF

e remuneração não são importados. A UF permanece nula em Aposentados, pois o campo não existe nessa fonte.

O banco possui as tabelas `servidor`, `cargo`, `orgao` e `uf`. A importação usa uma tabela temporária e exige o destino vazio. Os índices e a extensão `pg_trgm` apoiam as buscas por igualdade e similaridade. Os arquivos são organizados em duas pastas, uma por conjunto. Eles podem ser copiados de uma coleta existente ou baixados por um script opcional com duas listas de endereços HTTP fixos. Na primeira inicialização, o backend verifica se o banco está vazio, trata todos os meses dessas pastas e importa o CSV resultante. O arquivo intermediário é temporário. A aplicação não acessa o portal para baixar dados durante a inicialização.

Foram processados os 22 arquivos disponíveis de setembro de 2025 a julho de 2026, inclusive o arquivo vazio de dezembro. Os arquivos de agosto de 2026 não estavam disponíveis na coleta. O recorte, portanto, cobre as competências disponíveis no intervalo definido de setembro de 2025 a agosto de 2026.

Tabela 2. Tratamento dos arquivos disponíveis no período.

Fonte	Linhas lidas	Registros mantidos
Aposentados	6.392	6.377
Carreiras/Cargos	11.959.771	1.161.285
Total	11.966.163	1.167.662

A carga final contém 1.167.662 registros consultáveis, 3.033 cargos, 231 órgãos e 27 UFs. A comparação de nome, cargo, órgão e UF elimina a repetição da mesma combinação entre meses. Alterações nesses campos são mantidas; a contagem não equivale a pessoas distintas, pois o CPF não é usado como identificador. O volume nomeado do PostgreSQL preserva a carga quando os containers são recriados. Com uma carga existente, a preparação é ignorada. O Gunicorn só começa a atender após a importação terminar; o frontend aguarda a API ficar disponível. A ausência das fontes ou um tratamento sem registros válidos interrompem a preparação com uma mensagem de erro.

3.2. API e interface

O endpoint `/api/servidores` aceita nome exato, similaridade, cargo, UF, órgão e combinações. As respostas usam JSON e permitem paginação. A interface envia os filtros, mostra os resultados e apresenta mensagens durante a consulta, na ausência de registros e em caso de erro.

O Gunicorn usa um processo e quatro threads. O limite configurado é de duas buscas simultâneas, e o timeout SQL é de 2 segundos. A API devolve HTTP 429 quando o limite é excedido e HTTP 504 quando a consulta é cancelada por tempo. Entradas inválidas, rotas e páginas inexistentes também recebem erros em JSON.

3.3. Aplicação em funcionamento

As Figuras 2 e 3 mostram consultas reais pelo navegador, usando a carga de todo o período disponível, com API e PostgreSQL reais.

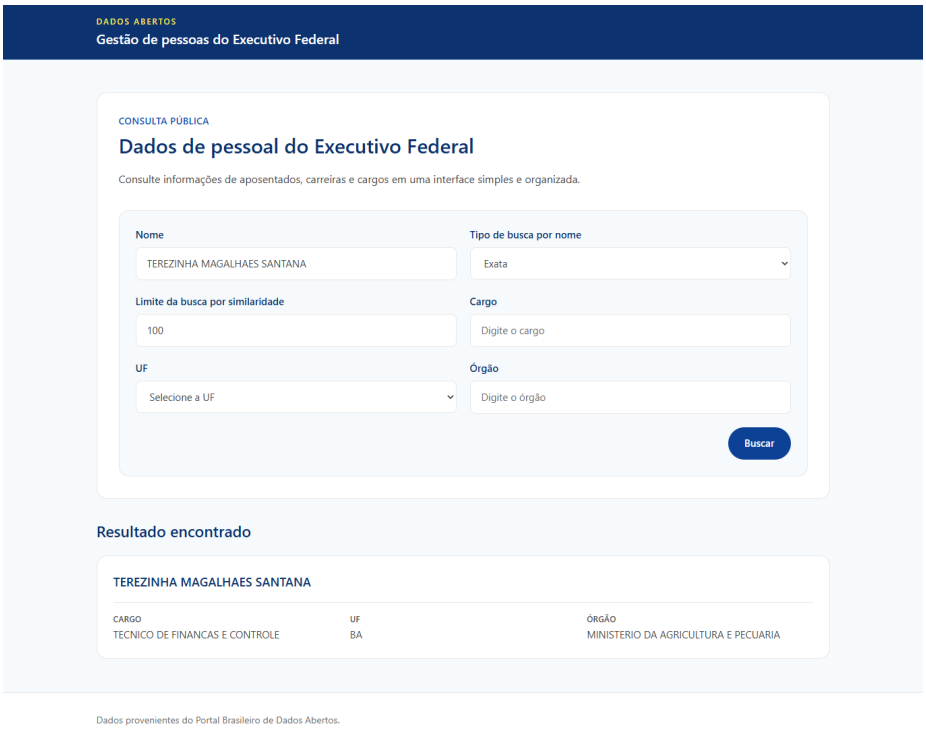


Figura 2. Busca por nome e apresentação de um único registro da base pública.

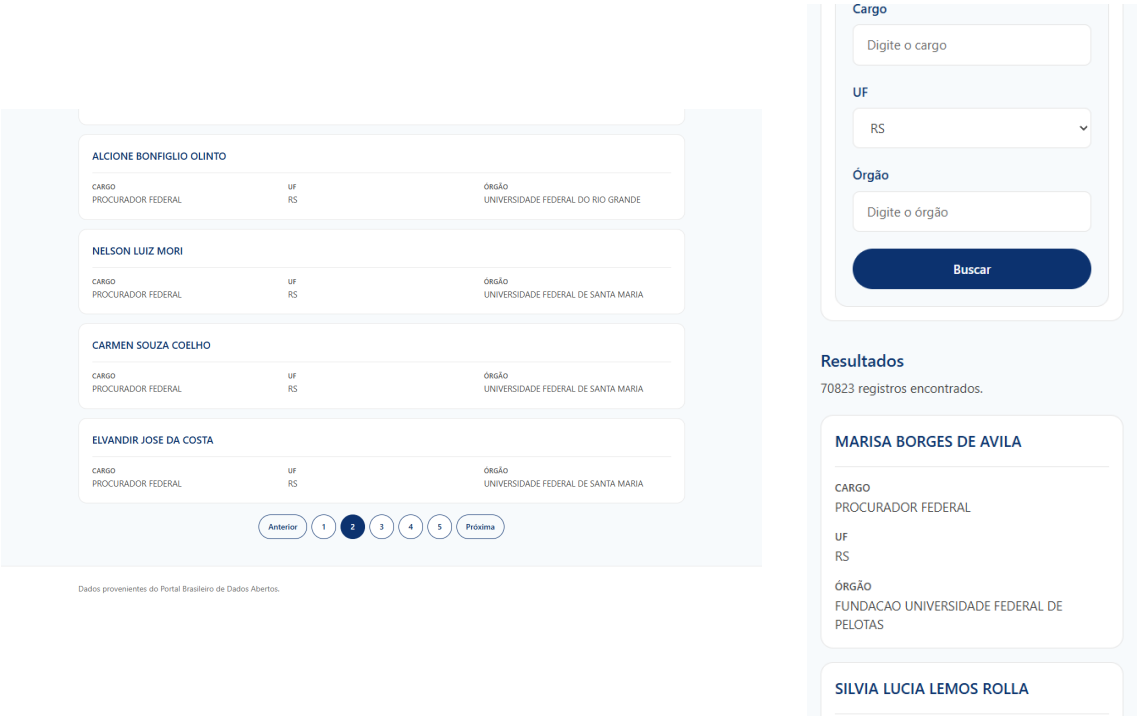


Figura 3. À esquerda, parte da segunda página de resultados. À direita, o resultado no celular.

3.4. Cliente desktop

O cliente adicional foi escrito em Python com Tkinter. A janela reúne os mesmos filtros da web: nome exato, similaridade com limite, cargo, UF, órgão e combinações. Os resultados são paginados em grupos de dez. Uma linha selecionada mostra os campos completos em texto copiável, incluindo a indicação de UF não informada quando o valor não existe na fonte.

As consultas usam a biblioteca padrão `urllib` e o endpoint HTTP da API, sem conexão direta ao banco. A requisição é executada em uma thread separada; a interface recebe o resultado por uma fila para não travar durante a espera. O endereço da API pode ser alterado na janela. O banco e o backend continuam necessários, mas o cliente não depende de Docker ou Python instalado.

O PyInstaller gerou dois executáveis x86_64, compilados separadamente nos sistemas de destino: Windows 10 e Ubuntu 22.04, com glibc 2.35. A Figura 4 mostra o cliente Tkinter executado pelos fontes no Windows, em consulta à mesma base usada pela aplicação web.



Figura 4. Cliente Tkinter no Windows: nome repetido nas fontes e indicação de UF não informada.

4. Testes e resultados

Os testes usaram PostgreSQL, API e interfaces em um ambiente local separado, com os arquivos completos disponíveis no período. A Tabela 3 reúne os resultados.

4.1. Consultas, proteção e persistência

Os cenários HTTP incluem filtros individuais e combinados, nomes repetidos, as três formas de curinga do enunciado, busca aproximada, paginação, limite de 100 registros,

Tabela 3. Resultados com a base de 1.167.662 registros.

Verificação	Resultado
Testes automatizados	54 aprovados: 15 de backend, 19 de dados e 20 de desktop e compilação.
Funcional e erros	26 cenários aprovados contra a API e o PostgreSQL.
SQL injection	45 casos; tabelas, contagens e fingerprints inalterados.
Carga de capacidade	20 requisições, concorrência 2, todas HTTP 200; 0,676 s no total e p95 de 98,03 ms.
Sobrecarga	200 requisições, concorrência 32; 8 HTTP 200 e 192 HTTP 429.
Saúde após a carga	`/health` respondeu HTTP 200.
Dependências do frontend	`npm audit` não apontou vulnerabilidades conhecidas após a atualização pontual do Vite.

entradas inválidas e ausência de resultados. O teste de timeout bloqueou temporariamente a tabela da stack de teste: a API respondeu HTTP 504 em 2,065 segundos e voltou a responder HTTP 200 após a liberação, sem alterar registros.

As tentativas de SQL injection não alteraram tabelas, contagens ou fingerprints do conteúdo do banco. Não houve erro de transporte nos testes de carga. As recusas por HTTP 429 mostram a proteção funcionando; não significam que o sistema atendeu 200 buscas simultâneas.

A conferência no navegador incluiu resultado único, consulta por UF, avanço de página e apresentação em tela de celular. A largura de 390 pixels não produziu rolagem horizontal. Não foram registradas exceções JavaScript. Depois de remover e recriar os containers de teste sem apagar o volume, os 1.167.662 registros permaneceram disponíveis, sem repetir a importação.

A inicialização foi exercitada em uma cópia limpa, com o volume do banco vazio e os 22 arquivos brutos disponíveis localmente. O próprio Compose executou o tratamento e a importação antes de liberar os serviços, sem comandos manuais de carga. Também foram verificadas as falhas por ausência de arquivos e por dados sem registros válidos. Com o banco preenchido, a aplicação iniciou sem precisar dos arquivos brutos e conservou a contagem de registros.

4.2. Desktop

Passaram 15 verificações com a janela Tkinter no Windows, executada pelos fontes contra a API real: filtros, combinações, similaridade, limite, paginação preservando os filtros enviados, UF ausente, busca sem resultados, limpeza e recuperação após erro de conexão. Os widgets e a rede eram reais.

Os executáveis Windows e Linux da entrega abriram e realizaram uma busca por nome na API real, com a base completa, retornando o registro esperado. No Windows, a consulta foi enviada pelo atalho Enter da própria janela. Os módulos embarcados da interface e do cliente HTTP também correspondem aos fontes testados, desconsiderando caminhos e posições de linhas. A execução Linux usou Ubuntu 22.04 com Xvfb, sem instalar Python ou Tkinter no ambiente de execução do cliente.

4.3. Limitações

Antes da primeira execução, é necessário baixar e organizar os arquivos públicos. O tratamento e a importação são executados na inicialização. A indisponibilidade de uma competência na fonte limita o período efetivamente carregado. A ausência de UF em Aposentados é uma limitação da fonte, apresentada como "Não informada", sem inventar um valor para completar o campo.

Os testes foram locais e não incluíram ensaios prolongados. O desktop foi compilado para Windows e Linux x86_64. Não foram validados ARM, todas as distribuições Linux, Wayland nativo, leitores de tela ou outras escalas de DPI. O cliente precisa alcançar a API em execução. O volume nomeado protege a carga na recriação dos containers, mas não substitui backup.

5. Avaliação do trabalho em equipe

A divisão entre infraestrutura, banco/API e interface ajudou a equipe a trabalhar em paralelo. Cada integrante manteve uma área principal, e a integração permitiu conferir se os filtros enviados pela tela eram entendidos pela API e se os dados importados apareciam corretamente.

A parte que mais consumiu tempo foi trabalhar com os datasets: localizar os arquivos, entender o tratamento necessário e organizar o que entraria nos scripts. Aposentados não tinha cabeçalho e trazia bytes NUL. Os arquivos de Carreiras/Cargos tinham linhas iniciais e delimitadores diferentes. Foi necessário conferir os dicionários e testar a leitura antes de montar a carga no banco. Meses indisponíveis e a ausência de UF em Aposentados também limitaram o que podia ser consultado.

As entregas se concentraram em alguns dias, e a documentação ficou mais para o final. Isso tornou mais trabalhosa a conferência do que já havia sido feito. Os testes integrados ajudaram a separar problemas da fonte de dados de problemas da aplicação e identificaram a divergência dos curingas, corrigida no backend. A proteção por concorrência evitou o acúmulo de buscas no teste de sobrecarga; em troca, parte dos acessos precisou ser recusada. Esse limite deve ser considerado ao apresentar a aplicação.

6. Entrega e conclusão

O ZIP contém uma pasta `projeto/`, com `backend/`, `frontend/`, `desktop/`, `db/`, Compose e documentação. Os executáveis do desktop são distribuídos separadamente para atender ao limite de tamanho do envio. Os arquivos públicos de dados e o banco preenchido ficam fora do pacote. O README apresenta a organização dos arquivos baixados e a execução por `docker compose up --build`. A primeira inicialização prepara e importa os arquivos locais; as seguintes reutilizam o banco preenchido.

Código-fonte e histórico de desenvolvimento no GitHub:
`https://github.com/arquivo-furg/spi2-consulta-servidores`.

Executáveis para Windows e Linux:
`https://github.com/arquivo-furg/spi2-consulta-servidores/releases/tag/entrega-final`.

O projeto reúne banco, API e interfaces web e desktop e registra as entregas dos três integrantes. As consultas, a paginação e a apresentação dos registros foram verificadas com dados sintéticos e com os arquivos completos disponíveis entre setembro de 2025 e agosto de 2026. O tratamento dos arquivos foi a principal dificuldade da equipe. Permanecem os limites de concorrência e de plataformas descritos neste relatório.

Referências

Docker (2026). Docker compose quickstart. `https://docs.docker.com/compose/gettingstarted/`. Acesso em: 9 out. 2026.

Ministério da Gestão e da Inovação em Serviços Públicos (2018). Dicionário de dados de aposentados. `https://repositorio.dados.gov.br/segrt/1.20180829-DadosRecursos-Aposentados.pdf`. Acesso em: 9 out. 2026.

Ministério da Gestão e da Inovação em Serviços Públicos (2019). Dicionário de dados de carreiras e cargos. `https://repositorio.dados.gov.br/segrt/2.20190709-DadosRecursos-Carreiras.pdf`. Acesso em: 9 out. 2026.

Portal Brasileiro de Dados Abertos (2026a). Gestão de pessoas do executivo federal – aposentados. `https://dados.gov.br/dados/conjuntos-dados/gestao-de-pessoas-executivo-federal--aposentados`. Acesso em: 9 out. 2026.

Portal Brasileiro de Dados Abertos (2026b). Gestão de pessoas do executivo federal – carreiras/cargos. `https://dados.gov.br/dados/conjuntos-dados/gestao-de-pessoas-executivo-federal---carreiras--cargos`. Acesso em: 9 out. 2026.

Sistemas para Internet II (2026). Trabalho do terceiro bimestre: Desenvolvimento de uma aplicação full stack. Enunciado da atividade, Universidade Federal do Rio Grande.